

# Nonlinear Optimization

Prof. Dr. Thomas Winter

Version 21. Oktober 2025

(Unconstrained) Nonlinear Optimization



## Overview

- The optimization problem
- Necessary conditions
- Sufficient conditions
- Gradient descent
- Step length (Armijo-Goldstein-step-length)
- Gradient descent using the Armijo-Goldstein-step-length
- Newton descent
- The trust region method
- Conjugate gradient descent



## Maximization of or Minimization?

Unless noted otherwise, we will focus on **minimization problems** only.

Every maximization problem can be easily transformed into a minimization problem since

$$\max f(\vec{x}) = -\min(-f(\vec{x}))$$

Since a maximization problem can be modelled as a minimization problem, we focus on minimization problems unless otherwise stated:

$$\min_{\vec{x} \in D \subseteq \mathbb{R}^n} f(\vec{x})$$



## Assumptions and notation

- Assumption:  $f$  is sufficiently often continuously differentiable on its definition domain  $D$ .

- Notation:

- $f$  is called **objective function**
- $\vec{x}^*$  is called **global minimum** if

$$f(\vec{x}^*) \leq f(\vec{x}) \quad \text{for all } \vec{x} \in D.$$

- $\vec{x}^*$  is called **strict global minimum** if  $f(\vec{x}^*) < f(\vec{x})$  for all  $\vec{x} \in D$ .
- $\vec{x}^*$  is called **local minimum** if  $f(\vec{x}^*) \leq f(\vec{x})$  for all  $\vec{x}$  in a neighborhood  $U_\delta(\vec{x}^*)$  of  $\vec{x}^*$  where  $U_\delta(\vec{x}^*) = \{\vec{x} \in D \mid \|\vec{x} - \vec{x}^*\| < \delta\}$ .
- $\vec{x}^*$  is called **strict local minimum** if the inequality is  $<$  strictly valid for all  $\vec{x}$  in the neighborhood  $U_\delta(\vec{x}^*)$  where  $\vec{x} \neq \vec{x}^*$ .



## Definition

- Necessary conditions are conditions that must be satisfied by a local minimum.

Example: The first derivative of  $f$ , the **gradient** of  $f$ , needs to be equal to  $\vec{0}$  at the point  $\vec{x} \in \mathbb{R}^n$  if  $\vec{x}$  is a local minimum of  $f$ .

$$\text{grad}f(\vec{x}) = \nabla f(\vec{x}) = \begin{pmatrix} \frac{\partial f}{\partial x_1} \\ \vdots \\ \frac{\partial f}{\partial x_n} \end{pmatrix} = \vec{0}$$

- A point  $\vec{x}^*$  with  $\nabla f(\vec{x}^*) = \vec{0}$  is called **stationary point** of  $f$ .



### Definition

- Sufficient conditions are conditions with which one can check if a given  $\vec{x}$  is a local minimum of  $f$ .

Example: If for a point  $\vec{x}^*$  the gradient of  $f$  is equal to  $\vec{0}$  and in addition the second derivative, the **Hessian matrix** of  $f$ , is **positive definite** then  $\vec{x}^*$  is a local minimum of  $f$ .

$$H_f(\vec{x}^*) = \left( \frac{\partial^2 f}{\partial x_i \partial x_j} \right)_{ij}$$



## Remark

- Please note that the condition that the gradient of  $f$  is equal to  $\vec{0}$  and  $H_f(\vec{x}^*)$  is positive definite is only sufficient but not necessary. A point  $\vec{x}^* \in D$  can be a local minimum of  $f$  although the Hessian matrix is not positive definite at  $\vec{x}^*$ .
- Example: The function  $f(x, y) = x^2 + y^4$  has a strict local (and global) minimum at  $\vec{x}^* = (x, y)^T = (0, 0)^T$ . However, the Hessian matrix of  $f$  is not positive definite!

$$H_f(x, y) = \begin{pmatrix} 2 & 0 \\ 0 & 12y^2 \end{pmatrix} \quad \text{hence} \quad H_f(0, 0) = \begin{pmatrix} 2 & 0 \\ 0 & 0 \end{pmatrix}$$

$H_f$  is **semi**-positive definite! (Eigenvalues are 2 and 0 (diagonal elements of a diagonal matrix).)



- If  $f$  is continuously differentiable then

$$f(\vec{x}) = f(\vec{x}^*) + \nabla f(\vec{x}^*)^T (\vec{x} - \vec{x}^*) + R(\vec{x}, \vec{x}^*) \quad \text{where} \quad \lim_{\vec{x} \rightarrow \vec{x}^*} \frac{R(\vec{x}, \vec{x}^*)}{\|\vec{x} - \vec{x}^*\|} = 0$$

- $f$  can be approximated by the following linear function

$$f(\vec{x}) = f(\vec{x}^*) + \nabla f(\vec{x}^*)^T (\vec{x} - \vec{x}^*)$$

- If  $f$  is two-times continuously differentiable then

$$f(\vec{x}) = f(\vec{x}^*) + \nabla f(\vec{x}^*)^T (\vec{x} - \vec{x}^*) + \frac{1}{2} (\vec{x} - \vec{x}^*)^T H_f(\vec{x}^*) (\vec{x} - \vec{x}^*) + R(\vec{x}, \vec{x}^*)$$

- $f$  can be approximated by the following quadratic function

$$f(\vec{x}) = f(\vec{x}^*) + \nabla f(\vec{x}^*)^T (\vec{x} - \vec{x}^*) + \frac{1}{2} (\vec{x} - \vec{x}^*)^T H_f(\vec{x}^*) (\vec{x} - \vec{x}^*)$$





## Necessary condition of second order

- If  $f$  is two-times continuously differentiable and if  $\vec{x}^*$  is a local minimum of  $f$  then  $H_f(\vec{x}^*)$  is **positive semi-definite**.
- A matrix  $A$  is **positive semi-definite** if  $\vec{d}^T A \vec{d} \geq 0$  for all  $\vec{d} \in \mathbb{R}^n, \vec{d} \neq \vec{0}$ .
- A symmetric matrix is positive semi-definite if all its eigenvalues are non-negative ( $\geq 0$ ). The eigenvalues of a matrix can be computed based the characteristic polynomial. In Octave/Matlab one can use the function `eig(A)`. In scipy you can use `scipy.linalg.eigvals(A)`.



## Theorem

Let

- $f$  be two-times continuously differentiable
- $\vec{x}^*$  be a stationary point of  $f$ , i.e.  $\text{grad} f(\vec{x}^*) = \vec{0}$ .
- $H_f(\vec{x}^*)$  be a positive definite matrix

Then  $\vec{x}^*$  is a strict local minimum of  $f$ .

Remark: As we observed for  $f(x, y) = x^2 + y^4$  this condition is not necessary!



- A function  $f : D \rightarrow \mathbb{R}$  is called **convex** if

$$f(\lambda \vec{x} + (1 - \lambda) \vec{y}) \leq \lambda f(\vec{x}) + (1 - \lambda) f(\vec{y})$$

for all  $\vec{x}, \vec{y} \in D$  and all  $\lambda \in [0, 1]$ .

This means that the line connecting the points  $(\vec{x}, f(\vec{x}))$  and  $(\vec{y}, f(\vec{y}))$  is not below (i. e. above) the graph of  $f$ .

- $f$  is called **strict convex** if the above inequality holds strictly for all  $\vec{x}, \vec{y} \in D$  and all  $\lambda \in (0, 1)$ .

### Theorem

- Every local minimum of a convex function  $f$  is a global minimum.
- If  $f$  is strict convex then there is at most one global minimum of  $f$ .
- If  $f$  is continuously differentiable then every stationary point of  $f$  is a global minimum.
- The maximum of a convex function is at the boundary of the definition domain.



## Descent directions and sufficient criterion of first order

- $\vec{d}$  is called **descent direction** of  $f$  in  $\vec{x}$  if there is a  $\delta > 0$  with

$$f(\vec{x} + c\vec{d}) < f(\vec{x}) \quad \forall c : 0 < c < \delta$$

- $\vec{d}$  is a descent directions of  $f$  in  $\vec{x}$  if  $\nabla f(\vec{x})^T \vec{d} < 0$  is.
- This condition means geometrically that the angle between the gradient and the descent direction is between  $90^\circ$  and  $270^\circ$ . (This follows from the fact that the cosine of these angles and the scalar product of such vectors is negative.)
- This condition is sufficient but not necessary. There may be descent directions with positive scalar product with the gradient.
- If  $\vec{x}^*$  is a local minimum then there is no descent direction, i. e.  $\nabla f(\vec{x}^*) = \vec{0}$ .



## Algorithm: generic descent method

- 1 Choose an initial point  $\vec{x}_0 \in D$ . Set the iteration index  $k$  to 0.
- 2 If a pre-defined stopping criterion is satisfied: STOP.
- 3 Compute a descent direction  $\vec{d}_k$  and a step size  $\alpha_k > 0$  so that

$$f(\vec{x}_k + \alpha_k \vec{d}_k) < f(\vec{x}_k)$$

- 4 Set  $\vec{x}_{k+1} = \vec{x}_k + \alpha_k \vec{d}_k$ .
- 5 Set  $k = k + 1$  and go to 2.)

- For steepest (gradient) descent  $\vec{d}_k = -\nabla f(\vec{x}_k)$ . It is a descent direction if  $\nabla f(\vec{x}_k) \neq \vec{0}$ .
- For Newton descent  $\vec{d}_k = -H_f(\vec{x}_k)^{-1} \cdot \nabla f(\vec{x}_k)$ . It is a descent direction if  $H_f$  is positive definite and if  $\nabla f(\vec{x}_k) \neq 0$



## General procedure for step size calculation

$$\min_{\vec{x} \in D \subseteq \mathbb{R}^n} f(\vec{x})$$

- We assume that  $f$  is sufficiently often continuously differentiable. Let  $D$  be an open set.
- Check if  $\vec{d}_k$  is a descent direction in the  $k$ -th iteration

$$\nabla f(\vec{x}_k)^T \cdot \vec{d}_k < 0$$

- How:

$$\varphi(\alpha) := f(\vec{x}_k + \alpha \cdot \vec{d}_k)$$

Compute the potential next function value  $\varphi(\alpha)$  to the current function value  $\varphi(0) = f(\vec{x}_k)$ . If  $\vec{d}_k$  is a descent direction, the function value should be smaller.



### Armijo-Goldstein step size

- Armijo-Goldstein step size  $\alpha_k$  in the  $k$ -th iteration

$$\alpha_k := \max\{\beta^j \mid j = 0, 1, 2, \dots\}$$

where  $\varphi(\alpha_k) \leq \varphi(0) + \sigma \cdot \alpha_k \cdot \varphi'(0)$ .

- $\varphi(0)$  is the current function value at  $\vec{x}_k$  and  $\varphi(\alpha_k)$  is the new function value after performing the step  $\vec{d}_k$  with step size  $\alpha_k$ .
- Initial choice of the parameters  $\sigma = 0.01$  and  $\beta = 0.9$  (default values).



### Armijo-Goldstein step size

The Armijo-Goldstein rule

1 Choose  $\beta \in (0, 1)$  and  $\sigma \in (0, 1)$ . Set  $\alpha := 1$ .

2 If

$$\varphi(\alpha) \leq \varphi(0) + \sigma \cdot \alpha \cdot \varphi'(0)$$

the set  $\alpha_k := \alpha$ . Stop.

3 Set  $\alpha := \beta \cdot \alpha$  and go to 2.)





### Computing the locally optimum step size

Instead of iterating and comparing based on  $\varphi$  whether or not a decrease in the objective function has been achieved one can minimize  $\varphi(\alpha)$  (in one variable  $\alpha$ ).

$$\alpha_k^* = \arg \min_{\alpha} \varphi(\alpha)$$

This step size is locally the best one. However, it is not guaranteed that it is better than the Armijo step size in terms of computation time and in the number of iterations necessary.



### Gradient descent using the Armijo-Goldstein step size

- 1 Choose an initial start point  $\vec{x}_0 \in \mathbb{R}^n$ . Set  $k = 0$ .
- 2 If a stopping criterion is satisfied (e. g.  $\nabla f(\vec{x}_i) = \vec{0}$ ): STOP.
- 3 Compute  $\vec{d}_k = -\nabla f(\vec{x}_k)$  and a step size  $\alpha_k$  using the Armijo-Goldstein rule.
- 4 Set  $\vec{x}_{k+1} = \vec{x}_k + \alpha_k \cdot \vec{d}_k$ .
- 5 Set  $k = k + 1$  and go to 2.)



### Theorem

If  $f$  is a function  $f : \mathbb{R}^n \rightarrow \mathbb{R}$  which is continuously differentiable. For each limit point  $\vec{x}^*$  of the sequence  $(\vec{x}_k)$  obtained from steepest (gradient) descent it holds that  $\nabla f(\vec{x}^*) = 0$ .

### Speed of convergence

The speed of convergence of steepest (gradient) descent is quite slow (see Nocedal/Wright: Chapter 3, page 44).

Example:  $f(x, y) = x^2 + 10y^2$ .



## Stopping conditions

- Ideal would be

$$\nabla f(\vec{x}_k) = 0$$

- Almost unlikely in practice due to computational and numerical inaccuracies. Hence

$$\|\nabla f(\vec{x}_k)\| \leq \epsilon$$

where  $\epsilon \approx 10^{-r}$  and  $r$  level of accuracy.

- Additionally: Define a maximum number of iterations  $k_{\max}$ .
- Problem: Scaling of the function  $f$  (when  $f$  has very small or very large values).  
Alternatives:

- $f(\vec{x}_{k-1}) - f(\vec{x}_k) \leq \epsilon \cdot (1 + |f(\vec{x}_k)|)$
- $\|\vec{x}_{k-1} - \vec{x}_k\| \leq \sqrt{\epsilon} \cdot (1 + \|\vec{x}_k\|)$
- $\|\nabla f(\vec{x}_k)\| \leq \sqrt[3]{\epsilon} \cdot (1 + |f(\vec{x}_k)|)$



### Motivation of Newton descent method

- In accordance the Taylor's theorem a two-times continuously differentiable function  $f$  can be approximated by a quadratic function in a neighborhood of  $\vec{x}_k$ .
- We define the quadratic approximation by:

$$q(\vec{d}) := f(\vec{x}_k) + \nabla f(\vec{x}_k)^T \vec{d} + \frac{1}{2} \vec{d}^T H_f(\vec{x}_k) \vec{d}$$

- We minimize  $q$  instead of  $f$ .
- We obtain that

$$\frac{dq}{d\vec{d}} = \nabla f(\vec{x}_k) + H_f(\vec{x}_k) \vec{d} = 0$$

- If  $H_f(\vec{x})$  is positive definite then there is a unique minimum  $\vec{d}^*$ .



### Motivation of Newton descent method

- If in addition  $H_f(\vec{x})$  can be inverted then  $\vec{d}^*$  can be easily computed:

$$\vec{d}^* = -H_f^{-1}(\vec{x})\nabla f(\vec{x})$$

- If  $\nabla f(\vec{x}) \neq 0$  then  $\vec{d}^*$  is a descent direction of  $f$  in  $\vec{x}$ .
- $H_f(\vec{x})$  is not always positive definite. For Newton descent method it is sufficient that  $H_f(\vec{x})$  is invertible.



## Local Newton descent method

- The Newton step can be computed by solving a (nonlinear) system of equations:

$$\begin{aligned}H_f(\vec{x}_k)\vec{d}_k &= -\nabla f(\vec{x}_k) \\ \vec{x}_{k+1} &= \vec{x}_k + \vec{d}_k\end{aligned}$$

- - 1 Choose a (good) starting point  $\vec{x}_0$  and set  $k = 0$ .
  - 2 If the stopping condition is satisfied: STOP.
  - 3 Compute (if possible) the direction  $\vec{d}_k$  by solving system of equations in the Newton step.
  - 4 Set  $\vec{x}_{k+1} = \vec{x}_k + \vec{d}_k$ .
  - 5 Set  $k = k + 1$  and go to 2.)



## Convergence of the local Newton descent method

### ■ Convergence

- We assume that  $f$  is 2-times continuously differentiable having a local minimum in  $\vec{x}^*$  and that  $H_f(\vec{x}^*)$  is invertible. Then there is a neighborhood of  $\vec{x}^*$  so that the local Newton descent method converges super-linearly (fast) to  $\vec{x}^*$  for all starting points in this neighborhood.
- If  $H_f$  satisfies the local Lipschitz-condition (i.e.  $H_f$  is of bounded variation)

$$\|H_f(\vec{x}) - H_f(\vec{y})\| \leq L\|\vec{x} - \vec{y}\|,$$

then the rate of convergence is quadratic (even faster).





The global Newton descent method combines steepest gradient descent and the local Newton descent method.

- 1 Choose a starting point  $\vec{x}_0 \in \mathbb{R}^n$  and choose  $\sigma \in (0, \frac{1}{2})$ ,  $\beta \in (0, 1)$ ,  $\rho > 0$  as well as  $p > 2$ . Set  $k = 0$ .
- 2 If the stopping condition is satisfied: STOP.
- 3 Set  $\alpha_k = 1$ .
- 4 Compute (if possible) the direction  $\vec{d}_k$  by solving system of equations in the Newton step. If this is impossible or if the condition

$$\nabla f(\vec{x}_k)^T \vec{d}_k \leq -\rho \cdot \|\vec{d}_k\|^p$$

is not satisfied then set  $\vec{d}_k = -\nabla f(\vec{x}_k)$  and compute the Armijo-Goldstein step size  $\alpha_k$ .

- 5 Set  $\vec{x}_{k+1} = \vec{x}_k + \alpha_k \vec{d}_k$ .
- 6 Set  $k = k + 1$  and go to 2.)



### Convergence of the global Newton descent method

- We assume that  $f$  is 2-times continuously differentiable having a local minimum in  $\vec{x}^*$  and that  $H_f(\vec{x}^*)$  is invertible.
- We denote by  $(\vec{x}_k)$  the sequence of solutions obtained by the global Newton descent method. Let  $x^*$  be a limit point of this sequence where  $H_f(\vec{x}^*)$  is positive definite.
- Then  $(\vec{x}_k)$  converges super-linearly (fast) to  $x^*$ .
- $x^*$  is a strict local minimum of  $f$ .
- If  $k$  is sufficiently large then  $\vec{d}_k$  is the Newton step for all subsequent  $k$  and the corresponding step size is  $\alpha_k = 1$ .
- If  $H_f(x^*)$  satisfies a (local) Lipschitz condition then  $(\vec{x}_k)$  converges to  $x^*$  with quadratic rate of convergence.



**BFGS-Update of  $H_f$  (Broyden-Fletcher-Goldfarb-Shanno)**

Instead of computing in each step the actual Hessian of  $f$  in  $\vec{x}_k$  approximate  $H_f$  by

$$H_{k+1}^{BFGS} = H_k + \frac{\vec{y}_k \vec{y}_k^T}{\vec{d}_k^T \vec{y}_k} - \frac{H_k \vec{d}_k \vec{d}_k^T H_k}{\vec{d}_k^T H_k \vec{d}_k}$$

where  $\vec{y}_k = \text{grad}f(\vec{x}_{k+1}) - \text{grad}f(\vec{x}_k)$  and  $\vec{d}_k = \vec{x}_{k+1} - \vec{x}_k$ .

**DFP-Update of  $H_f$  (Davidon-Fletcher-Powell)**

Instead of computing in each step the actual Hessian of  $f$  in  $\vec{x}_k$  approximate  $H_f$  by

$$H_{k+1}^{DFP} = H_k + \frac{(\vec{y}_k - H_k \vec{d}_k) \vec{y}_k^T + \vec{y}_k (\vec{y}_k - H_k \vec{d}_k)^T}{\vec{y}_k^T \vec{d}_k} - \frac{(\vec{y}_k - H_k \vec{d}_k)^T \vec{d}_k}{(\vec{y}_k^T \vec{d}_k)^2} \vec{y}_k \vec{y}_k^T$$

where  $\vec{y}_k = \text{grad}f(\vec{x}_{k+1}) - \text{grad}f(\vec{x}_k)$  and  $\vec{d}_k = \vec{x}_{k+1} - \vec{x}_k$ .



**Idea: Approximate  $f$  by its Taylor expansion of second order**

$f$  can be approximated at  $\vec{x}_k$  by the following quadratic function

$$q(\vec{d}) = f(\vec{x}_k) + \nabla f(\vec{x}_k)^T \vec{d} + \frac{1}{2} \vec{d}^T H_f(\vec{x}_k) \vec{d}$$

where  $\vec{d} = \vec{x}_{k+1} - \vec{x}_k$ .

This approximation is not necessarily good but it should be good in a local neighborhood of  $\vec{x}_k$  called **trust region**.



- 1 Choose  $\vec{x}^0 \in \mathbb{R}^n$ ,  $\Delta_0$  and  $\epsilon > 0$ . Set  $k = 0$ .
- 2 Compute  $g^k := \nabla f(\vec{x}^k)$ . If  $|g^k| < \epsilon$ : Stop!
- 3 Compute  $H_k := H_f(\vec{x}^k)$ .
- 4 Compute  $\vec{d}^k \in \mathbb{R}^n$  by optimizing the quadratic function  $\min q_k(\vec{d})$  subject to  $|\vec{d}| \leq \Delta_k$ .

$$r_k := \frac{f(\vec{x}^k) - f(\vec{x}^k + \vec{d}^k)}{q_k(0) - q_k(\vec{d}^k)}$$

$$\Delta_{k+1} := \begin{cases} \frac{1}{4}\Delta_k & \text{if } r_k < \frac{1}{4} \\ 2\Delta_k & \text{if } r_k > \frac{3}{4} \text{ and } |\vec{d}^k| = \Delta_k \\ \Delta_k & \text{otherwise} \end{cases} \begin{array}{l} \text{(reduce TR)} \\ \text{(expand TR)} \\ \text{(keep TR)} \end{array}$$

- 5 If  $r_k > \frac{1}{4}$  set  $\vec{x}^{k+1} := \vec{x}^k + \vec{d}^k$  (perform the step)  
 Otherwise set  $\vec{x}^{k+1} := \vec{x}^k$  (repeat the computation in a smaller region).  
 Set  $k = k + 1$  and goto 2.)



## Idea

The idea of the conjugate gradient method is based on the minimization of a quadratic function

$$f(\vec{x}) = \frac{1}{2} \vec{x}^T A \vec{x} - \vec{x}^T \vec{b}$$

by iteratively solving a system of linear equations  $A\vec{x} = \vec{b}$ . In each step the current residuum (the current error) is computed:

$$\text{grad} f(\vec{x}_k) = A\vec{x}_k - \vec{b} =: -\vec{r}_k$$

The choice of the next direction  $\vec{d}_k$  shall be conjugated to all previous steps, i.e.

$$\vec{d}_{k+1}^T A \vec{d}_k = 0$$



## Conjugate gradient method for quadratic functions

0. Choose a starting point  $\vec{x}_0 \in \mathbb{R}^n$  and set  $k = 0$ .  
Compute  $\vec{r}_0 = \vec{b} - A\vec{x}_0$  and set  $\vec{d}_0 = \vec{r}_0$ .
1. Compute  $\vec{z} = A\vec{d}_k$ .
2. Compute  $\alpha_k = \frac{\vec{r}_k^T \vec{r}_k}{\vec{d}_k^T \vec{z}}$ .  
Set  $\vec{x}_{k+1} = \vec{x}_k + \alpha_k \vec{d}_k$ . Set  $\vec{r}_{k+1} = \vec{r}_k - \alpha_k \vec{z}$ .
3. Conjugate  $\vec{d}_{k+1}$  using  $\vec{d}_k$  and  $\vec{r}_k$ :  
Compute  $\beta_k = \frac{\vec{r}_{k+1}^T \vec{r}_{k+1}}{\vec{r}_k^T \vec{r}_k}$ .  
Set  $\vec{d}_{k+1} = \vec{r}_{k+1} + \beta_k \vec{d}_k$ .
4. If  $\|\vec{r}_{k+1}\| < \epsilon$ : stop. Otherwise go to 1.

There are different ways to compute  $\beta_k$ .

The conjugate gradient method converges for symmetric, positive definite matrices  $A$ .



## Conjugate gradient method for nonlinear functions

0. Choose a starting point  $\vec{x}_0 \in \mathbb{R}^n$  and set  $k = 0$ .  
Compute  $\vec{d}_0 = -\text{grad} f(\vec{x}_0)$ .  
Compute  $\alpha_0 = \arg \min_{\alpha} f(x_0 + \alpha d_0)$ .  
Set  $\vec{x}_1 = \vec{x}_0 + \alpha_0 \vec{d}_0$ .  
Set  $\vec{s}_0 = \vec{d}_0$ .
1. Compute  $\vec{d}_k = -\text{grad} f(\vec{x}_k)$ .  
If  $\|\vec{d}_k\| < \epsilon$ : stop.
2. Conjugate  $\vec{d}_k$  to the previous directions:  
Compute  $\beta_k = \frac{\|\vec{d}_k\|}{\|\vec{d}_{k-1}\|}$ .  
Set  $\vec{s}_k = \vec{d}_k + \beta_k \vec{s}_{k-1}$ .
3. Compute  $\alpha_k = \arg \min_{\alpha} f(x_k + \alpha s_k)$ .  
Set  $\vec{x}_{k+1} = \vec{x}_k + \alpha_k \vec{s}_k$ .  
Go to 1.

